

Real-Time Collaborative Traffic & Road Incident Reporting System

Operating Systems Lab Mini Project

April 30, 2026

Abstract

This project implements a real-time traffic incident reporting system using a multi-user client-server architecture. It demonstrates core operating system concepts such as concurrency control, file locking, socket programming, and inter-process communication. The system allows commuters, traffic officers, and control room operators to collaboratively report, monitor, and resolve incidents efficiently.

1 Introduction

Modern traffic systems require fast and coordinated responses. This system simulates a real-world environment where multiple users interact simultaneously with a central server. The design focuses on reliability, concurrency safety, and real-time communication.

2 Problem Statement

Modern urban traffic systems require efficient coordination between citizens, officers, and control authorities. Existing systems suffer from delayed communication, lack of centralized monitoring, and unsafe concurrent data handling.

The system aims to:

- Enable real-time incident reporting
- Support multiple concurrent users
- Ensure data consistency
- Enforce role-based access control

3 Key Features

- **Multi-User System:** Multiple clients operate simultaneously without interference.
- **Role-Based Access:** Each role (commuter, officer, control room) has restricted permissions.
- **Real-Time Processing:** Incidents are handled instantly through server communication.
- **Concurrency Safety:** Mutex and semaphores ensure safe access to shared data.
- **File Protection:** File locking prevents corruption during concurrent access.
- **Live Monitoring:** Analytics process displays real-time updates using IPC.
- **Scalable Design:** Thread-per-client model supports multiple users.

4 System Architecture

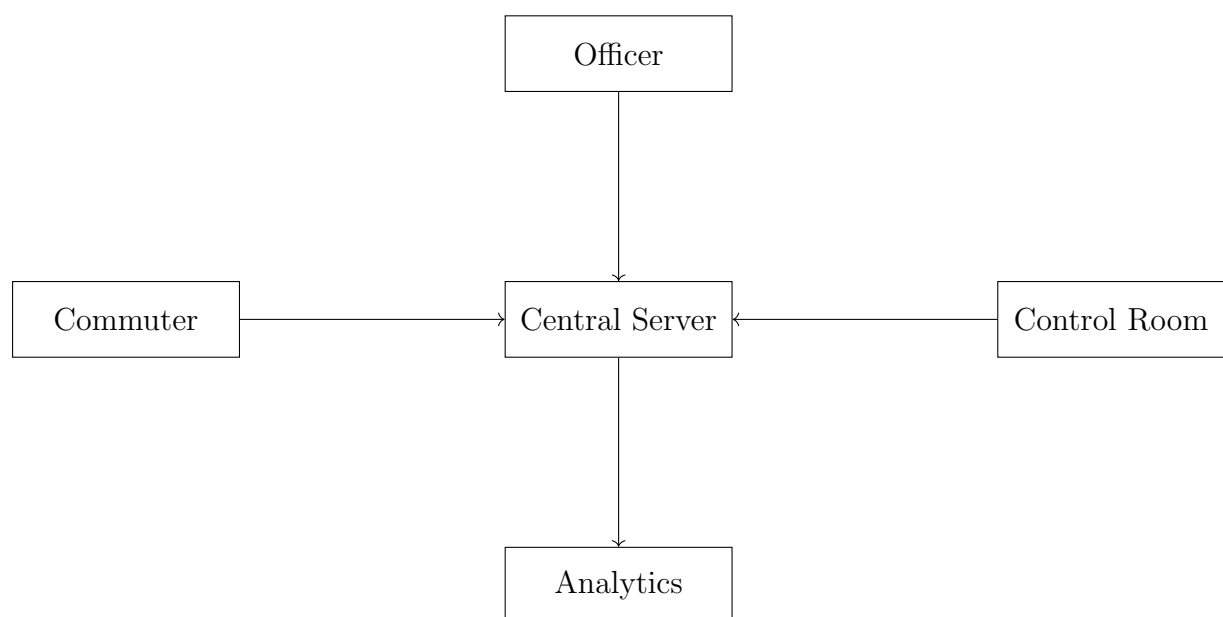


Figure 1: System Architecture

5 Operating System Concepts Implementation

5.1 Role-Based Authorization

Problem: Different users require different access permissions.

Implementation: The server authenticates users and assigns roles. Each command is validated based on role permissions.

Why: Ensures security and prevents unauthorized actions.

5.2 File Locking

Problem: Concurrent writes can corrupt log files.

Implementation: `fcntl()` is used to apply read/write locks on files.

Why: Guarantees safe file access.

5.3 Concurrency Control

Problem: Multiple threads accessing shared data cause race conditions.

Implementation:

- Mutex protects shared incident registry
- Semaphore limits concurrent file operations

Why: Ensures consistent and safe execution.

5.4 Data Consistency

Problem: Invalid updates can corrupt system state.

Implementation: Controlled state transitions: PENDING → INVESTIGATING → RESOLVED

Why: Maintains logical correctness.

5.5 Socket Programming

Implementation: TCP sockets enable communication between clients and server.

Why: Supports real-time distributed interaction.

5.6 Inter-Process Communication

Implementation: Named pipe (FIFO) connects server and analytics process.

Why: Enables real-time data sharing.

6 Data Flow

- Commuter reports incident
- Server processes request
- Officer claims and resolves
- Control room monitors system

7 State Transition Diagram



Figure 2: State Transition Diagram

8 Challenges Faced and Solutions

- **Race Conditions** Solved using mutex to ensure exclusive access.
- **File Corruption** Solved using file locking with `fcntl()`.
- **Synchronization Issues** Solved using semaphores to control concurrent operations.
- **Real-Time Updates** Solved using named pipes for IPC.

9 Conclusion

This project demonstrates practical implementation of operating system concepts through a real-world system. It ensures safe concurrency, reliable communication, and efficient coordination between multiple users.

10 Screenshots of Output

```
=== Traffic Analytics Process v2 ===
IPC channels: Named Pipe + Shared Memory
Waiting for server...
Connected! Receiving events...

TRAFFIC INCIDENT ANALYTICS v2 - DUAL IPC DASHBOARD
[PIPE] Last: CREATED Zone:1 Incident#1 Events:1
[SHM] Created:0 Resolved:0 Escalated:1 AvgResp:0
[SHM] Priority: CRIT:1 HIGH:0 MED:0 LOW:0

Zone | Created | Claimed | Resolved | Escalated | Pending
-----|-----|-----|-----|-----|-----
1 | 1 | 0 | 0 | 0 | 1

[Pipe: /tmp/traffic_analytics_pipe | ShmMem: /traffic_shm_stats]
```

```
Logged in as: ravi
-----
REPORT <zone> LOW|MEDIUM|HIGH|CRITICAL <desc>
LIST <zone>
STATS
QUIT
-----
> REPORT 1 MEDIUM Road Accident

OK Incident #1 [MEDIUM] created in zone 1

Logged in as: ravi
-----
REPORT <zone> LOW|MEDIUM|HIGH|CRITICAL <desc>
LIST <zone>
STATS
QUIT
-----
```

TRAFFIC INCIDENT SYSTEM | Officer Terminal

=====

Connecting to server... Connected.

Username : officer1

Password : pass1

Login successful. Assigned to Zone 1.

Officer : officer1

Zone : 1

LIST	View your zone incidents
CLAIM <id>	Claim an incident
UPDATE <id> INVESTIGATING	Mark as investigating
UPDATE <id> RESOLVED	Mark as resolved
REPORT <zone> <priority> <desc>	File new incident
STATS	View live statistics
QUIT	

=== Traffic System - Control Room v2 ===

Server: WELCOME Traffic Incident System v2. LOGIN <user> <pass>

Username: admin

Password: admin123

✓ AUTH_OK Welcome admin! Role:CONTROL_ROOM Zone:-1

Commands:

LISTALL | LIST <zone> | CLAIM <id>
UPDATE <id> INVESTIGATING|RESOLVED | CLOSE <id>
STATS (shared memory live stats)
ALERTS (drain escalation message queue)
REPORT <zone> CRITICAL <desc>
QUIT

Live escalation alerts will appear automatically.